Behavioral Synthesis of ICs

High Level Synthesis Algorithms 2

Credentials: majority of information are from the following sources:

1. Gajski, Daniel D., Nikil D. Dutt, Allen CH Wu, and Steve YL Lin. *High-Level Synthesis: Introduction to Chip and System Design*. Springer Science & Business Media, 2012.
2. Mohanty, Saraju P., Nagarajan Ranganathan, Elias Kougianos, and Priyardarsan Patra. *Low-power high-level synthesis for nanoscale CMOS circuits*. Springer Science & Business Media, 2008.

Carleton University

Outline

- Allocation
- Datapath Architectures
- Allocation Problem
  - Unit Selection
  - Functional-Unit Binding
  - Interconnection Binding
  - Storage Binding
  - Interconnection Binding
  - Interdependence and Ordering
- Greedy Constructive Approaches
- Decomposition Approaches
  - Clique Partitioning
- Graph Coloring Approach

Objectives

→ Understanding Binding and Allocation in high level synthesis

Allocation

Allocation

- As described in the previous lectures, scheduling assigns operations to control steps and thus converts a behavioral description into a set of register transfers that can be described by a state table.
- A target architecture for such a description is the FSMD. We derive the control unit for such a FSMD from the control-step sequence and the conditions used to determine the next control step in the sequence.
- The datapath is derived from the register transfers assigned to each control step; this task is called datapath synthesis or datapath allocation.
- A datapath in the FSMD model is a net list composed of three types of register transfer (RT) components or units: functional, storage and interconnection.
  - Functional units, such as adders, shifters, ALUs and multipliers, execute the operations specified in the behavioral description.
  - Storage units, such as registers, register files, RAMs and ROMs, hold the values of variables generated and consumed during the execution of the behavior.
  - Interconnection units, such as buses and multiplexers, transport data between the functional and storage units.

Allocation

→ Datapath allocation consists of two essential tasks:

- Unit selection,
- Unit binding.

→ Unit selection determines the number and types of RT components to be used in the design.

→ Unit binding involves the mapping of the variables and operations in the scheduled CDFG into the functional, storage and interconnection units, while ensuring that the design behavior operates correctly on the selected set of components.

- For every operation in the CDFG, we need a functional unit that is capable of executing the operation.
- For every variable that is used across several control steps in the scheduled CDFG, we need a storage unit to hold the data values during the variable's lifetime.
- For every data transfer in the CDFG, we need a set of interconnection units to effect the transfer.

Allocation

→ Besides the design constraints imposed on the original behavior and represented in the CDFG, additional constraints on the binding process are imposed by the type of hardware units selected.

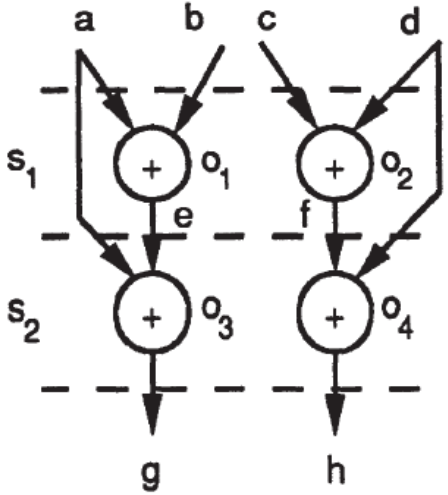
- For **example**, a functional unit can execute only one operation in any given control step.
- For **example**, the number of multiple accesses to a storage unit during a control step is limited by the number of parallel ports on the unit.

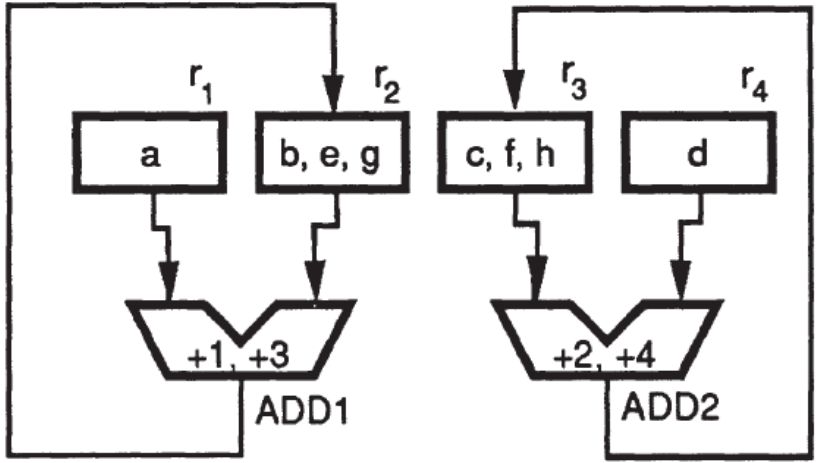
→ The mapping of variables and operations in the DFG of Figure 1 (see next slide) are illustrated into RT components.

- Let us select two adders, $ADD1$ and $ADD2$, and four registers, r_1, r_2, r_3 and r_4.
- Operations o_1 and o_2 cannot be mapped into the same adder because they must be performed in the same control step s_1.

Allocation

→ **Figure 1**: Mapping of behavioral objects into RT components.





Allocation

- On the other hand, operation o_1 can share an adder with operation o_3 because they are carried out during different control steps.
- Thus, operations o_1 and o_3 are both mapped into $ADD1$.
- Variables a and e must be stored separately because their values are needed concurrently in control step s_2.
- Registers r_1 and r_2, where variables a and e reside, must be connected to the input ports of $ADD1$; otherwise, operation o_3 will not be able to execute in $ADD1$.
- Similarly, operations o_2 and o_4 are mapped to $ADD2$.
- Note that there are several different ways of performing the binding.
- For **example**, we can map o_2 and o_3 to $ADD1$ and o_1 and o_4 to $ADD2$.

Allocation

→ Besides implementing the correct behavior, the allocated datapath must meet the overall design constraints in terms of the metrics defined before (e.g., area, delay, power dissipation, etc.).

→ To simplify the allocation problem, some algorithms use two quality measures for datapath allocation:

- the total size (i.e., the silicon area) of the design,
- the worst-case register-to-register delay (i.e., the clock cycle) of the design.

→ We can solve the allocation problem in different ways such as:

- greedy approaches, which progressively construct a design while traversing the CDFG;
- decomposition approaches, which decompose the allocation problem into its constituent parts and solve each of them separately;
- iterative methods, which try to combine and interleave the solution of the allocation subproblems.

Datapath Architectures

Datapath Architectures

→ A datapath architecture defines the characteristics of the datapath units and the interconnection topology.

→ A simple target architecture may greatly reduce the complexity of the synthesis problems since the number of alternative designs is greatly reduced.

→ On the other hand, a less constrained architecture, although more difficult to synthesize, may result in higher quality designs.

→ While an oversimplified datapath architecture leads to elegant synthesis algorithms, it also usually results in unacceptable designs.

An AI generated image for this argument!

Datapath Architectures

→ The interconnection topology that supports data transfers between the storage and functional units is one of the factors that has a significant influence on the datapath performance.

→ The complexity of the interconnection topology is defined by the maximum number of interconnection units between any two ports of functional or storage units.

→ Each interconnection unit can be implemented with a multiplexer or a bus.

→ For **example**, Figure 2 shows two datapaths, using multiplexer and bus interconnection units respectively, which implement the following five register transfers:

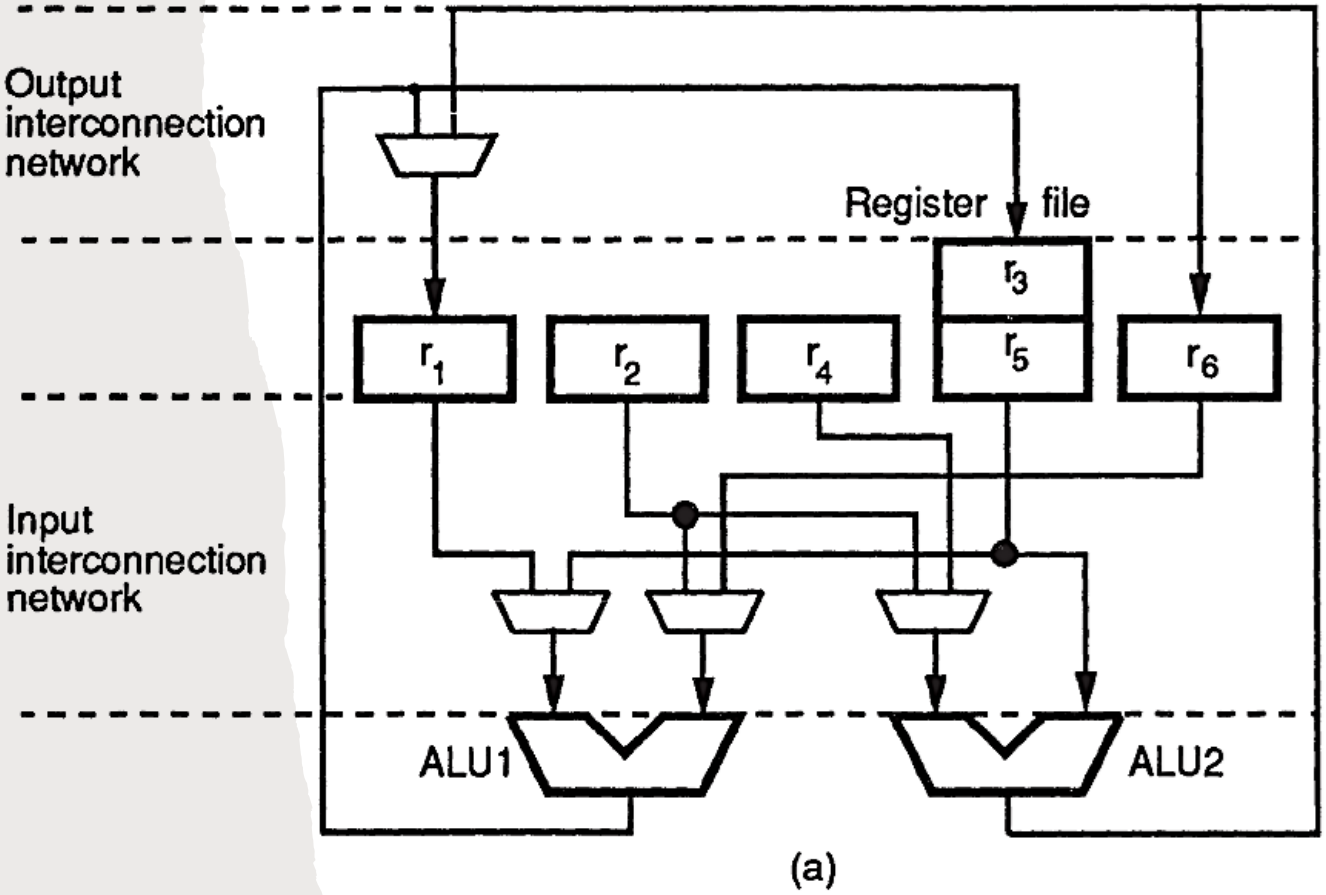
$$
\begin{aligned}
s_1&: r_3 \Leftarrow ALU1\,(r_1, r_2);\quad r_1 \Leftarrow ALU2\,(r_3, r_4);\\
s_2&: r_1 \Leftarrow ALU1\,(r_5, r_6);\quad r_6 \Leftarrow ALU2\,(r_2, r_5);\\
s_3&: r_3 \Leftarrow ALU1\,(r_1, r_6);\ .
\end{aligned}
$$

Datapath Architectures

Figure 2: Datapath interconnections:

a) a multiplexer-oriented datapath,

b) a bus-oriented datapath.



Datapath Architectures

→ We call the interconnection topology "point-to-point" if there is only one interconnection unit between any two ports of the functional and/or storage units.

→ The point-to-point topology simplifies the allocation algorithms. In this topology, we create a connection between any two functional or storage units as needed.

→ If more than one connection is assigned to the input of a unit, a multiplexer or a bus is used.

→ In order to minimize the number of interconnections, we can combine registers into register files with multiple ports. Each port may support read and/or write accesses to the data.

→ Some register files allow simultaneous read and write accesses through different ports.

→ Although register files reduce the cost of interconnection units, each port requires dedicated decoder circuitry inside the register file, which increases the storage cost and the propagation delay.

Datapath Architectures

→ To simplify the binding problem, here we assume that all register transfers go through functional units and that direct interconnections of two functional units are not allowed.

→ Therefore, we only need interconnection units to connect the output ports of storage units to the input ports of functional units (i.e., the input interconnection network) and the output ports of functional units to the input ports of storage units (i.e., the output interconnection network).

→ The complexity of the input and output interconnection networks need not be the same. One can be simplified at the expense of the other.

→ For **example**, selectors may not be allowed in front of the input ports of storage units.

→ This results in a more complicated input interconnection network, and, hence, an imbalance between the read and write times of the storage units.

Datapath Architectures

→ In addition to affecting the allocation algorithms, such an architecture increases testability.

→ Furthermore, the number of buses that each register file drives can be constrained. If a unique bus is allocated for each register-file output, tri-state bus drivers are not needed between the registers and the buses.

→ This restriction on register-file outputs produces more multiplexers at the input ports of functional units.

→ Moreover, some variables may need to be duplicated across different register files in order to simplify the selector circuits between the buses and the functional units.

Datapath Architectures

→ Another interconnection scheme commonly used in processors has the buses partitioned into segments so that each bus segment is used by one functional unit at a time.

→ The functional and storage units are arranged in a single row with bus segments on either side, and so looks like a 2-bus architecture.

→ A data transfer between the segments is achieved through switches between bus segments.

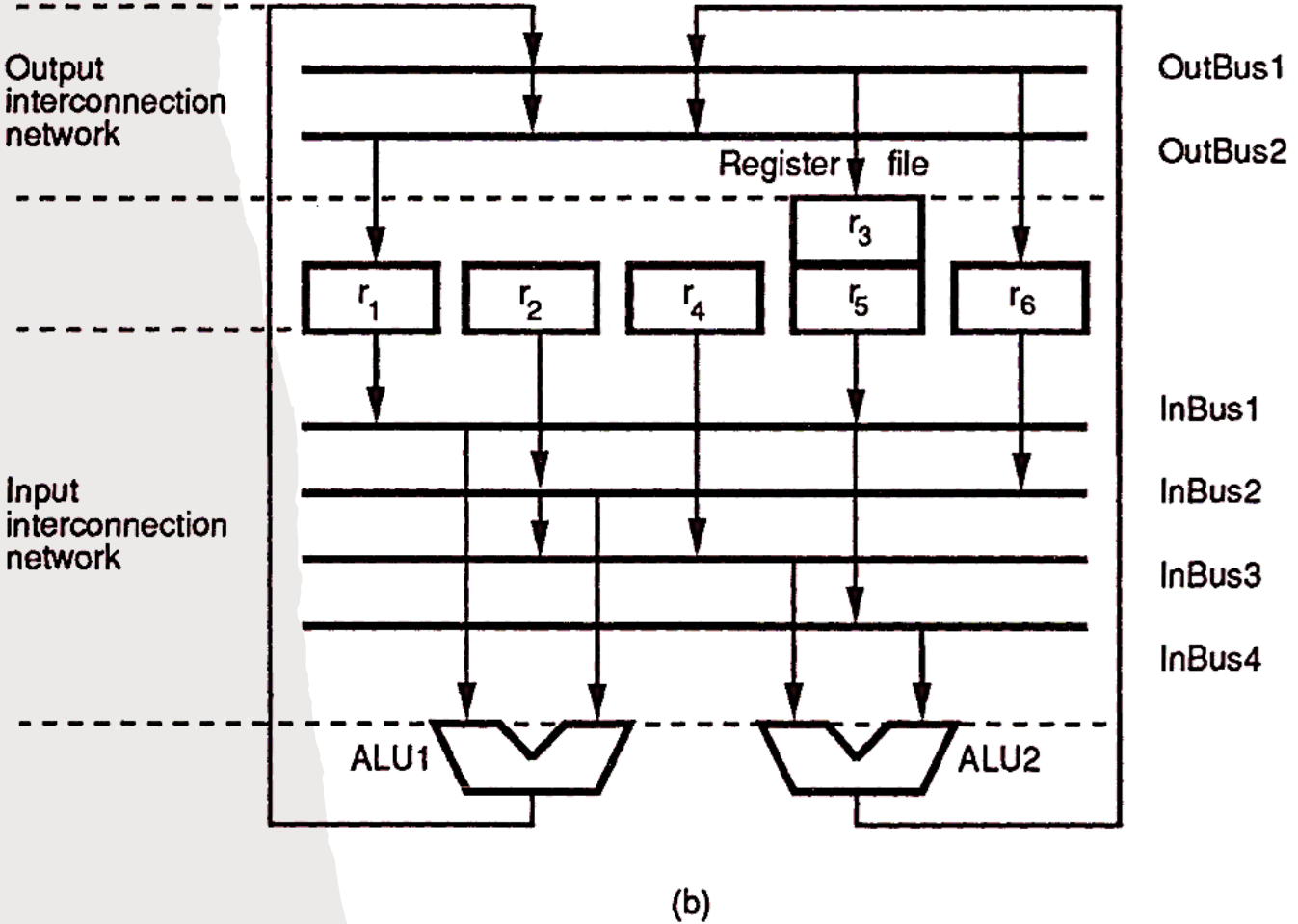
→ Thus, we accomplish interconnection allocation by controlling the switches between the bus segments to allow or disallow data transfers.

Datapath Architectures

Figure 2: Datapath interconnections:

a) a multiplexer-oriented datapath,

b) a bus-oriented datapath.



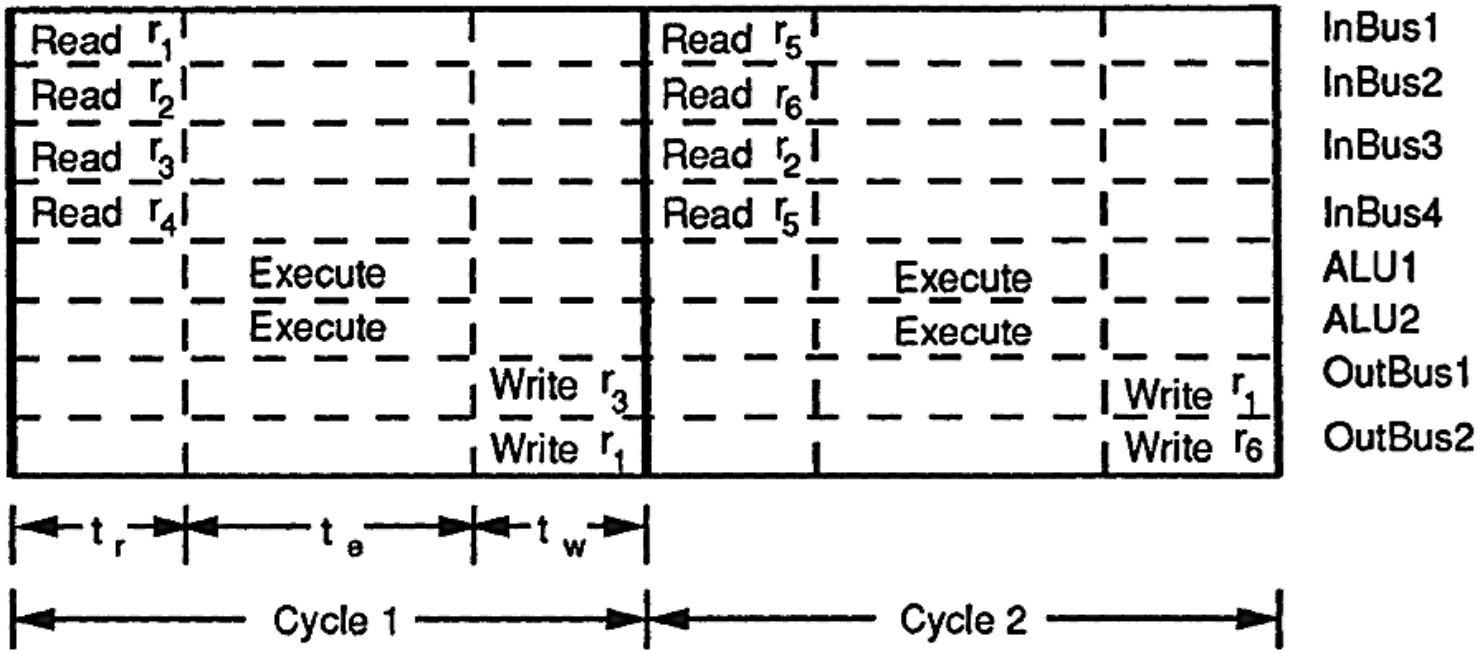
Datapath Architectures

→ Let us analyze the delays involved in register-to-register transfers for the five-transfer example in Figure 2.

- The relative timing of read, execute and write micro-operations in the first two clock cycles of the example are shown in Figure 3:
  - t_r is the time delay involved for reading the data out of the registers and then propagating through the input interconnection network;
  - t_e is the propagation delay through a functional unit;
  - t_w is the delay for data to propagate from the functional units through the output interconnection network and be written to the registers.
- In the target architectures described so far, all the components along the path from the registers' output ports back to the registers' input ports (i.e., the input interconnection network, the ALUs and the output interconnection network) are combinational.
- Thus, the clock cycle will be equal to or greater than $t_r + t_e + t_w$. ($t_{clock} \geq t_r + t_e + t_w$)

Datapath Architectures

→ **Figure 3**: Sequential execution of three micro-operations in the same clock period.



Datapath Architectures

→ In addition to the registers, latches also may be inserted at the input and/or output ports of functional units to improve the datapath performance.

→ When latches are inserted only at the outputs of the functional units (Figure 4), the read accesses and functional-unit execution for operations scheduled into the current control step can be performed at the same time as the write accesses of operations scheduled into the previous control step (Figure 5).

→ The cycle period is reduced to

$$MAX(tr + t_e, t_w).$$

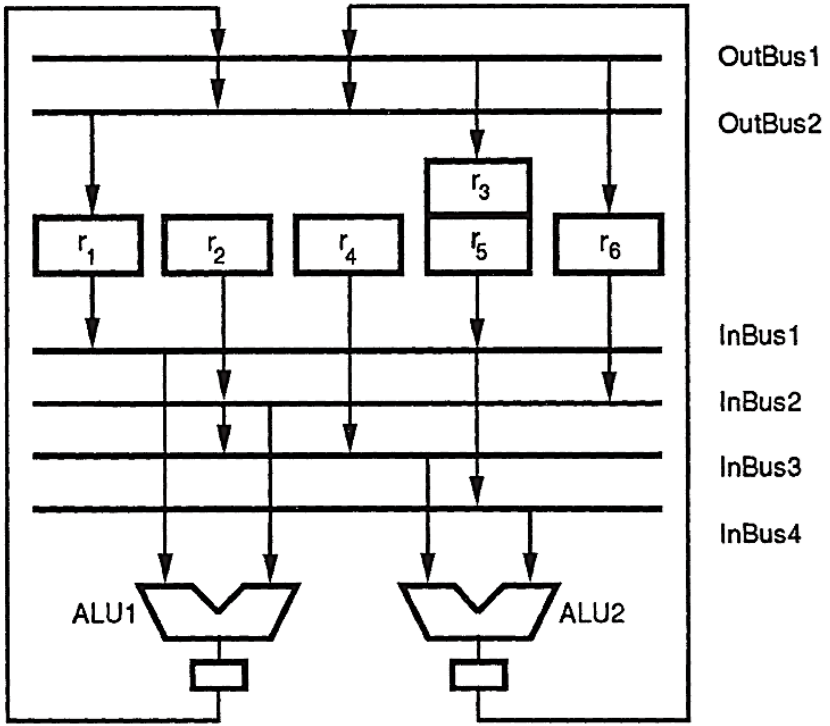
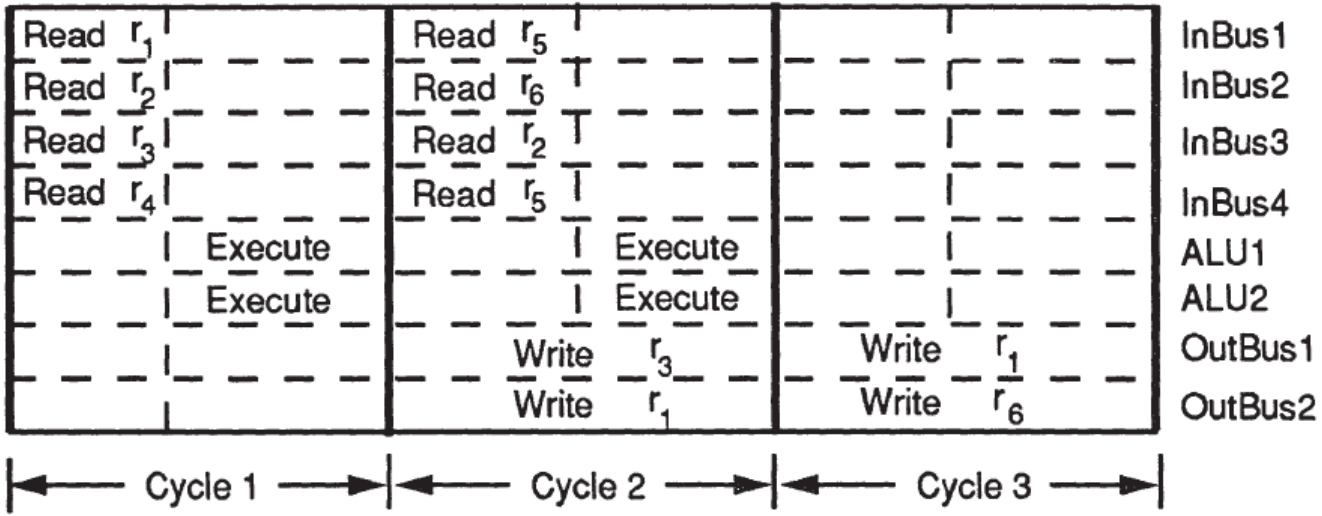


Figure 4: Insertion of latches at the output ports of the functional units.

Datapath Architectures

→ **Figure 5**: Overlapping read and write data transfers.



Datapath Architectures

→ But the register transfers are not well balanced:

- reading of the registers and execution of an ALU operation are performed in the first cycle,
- while only writing of the result back into the registers is performed during the second cycle.

→ Similarly, if only the inputs of the functional units are latched, the read accesses for operations scheduled into the next control step can be performed at the same time as the functional unit-execution and the write accesses for operations scheduled into the current control step.

→ The cycle period in that case will be

$$MAX(tr, t_e, +t_w).$$

→ In either case, the register files and latches are controlled by a single-phase clock.

Datapath Architectures

→ Operation execution and the reading/writing of data can take place concurrently when both the inputs and outputs of a functional unit are latched (See Figure 6).

→ The three combinational components, namely, the input-interconnection units, the functional units and the output interconnection units, can all be active concurrently.

→ Figure 7 shows how this pipelining scheme works.

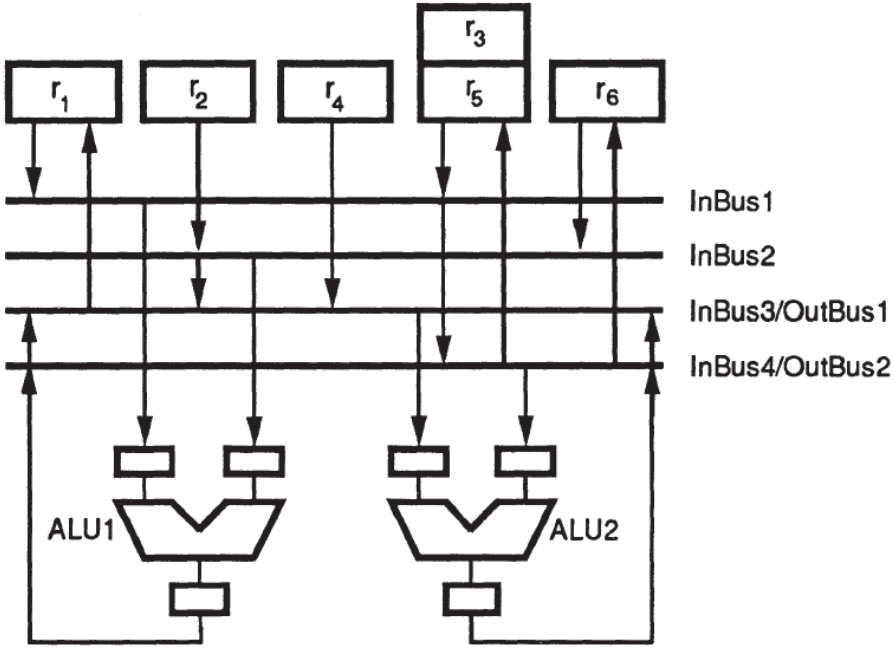
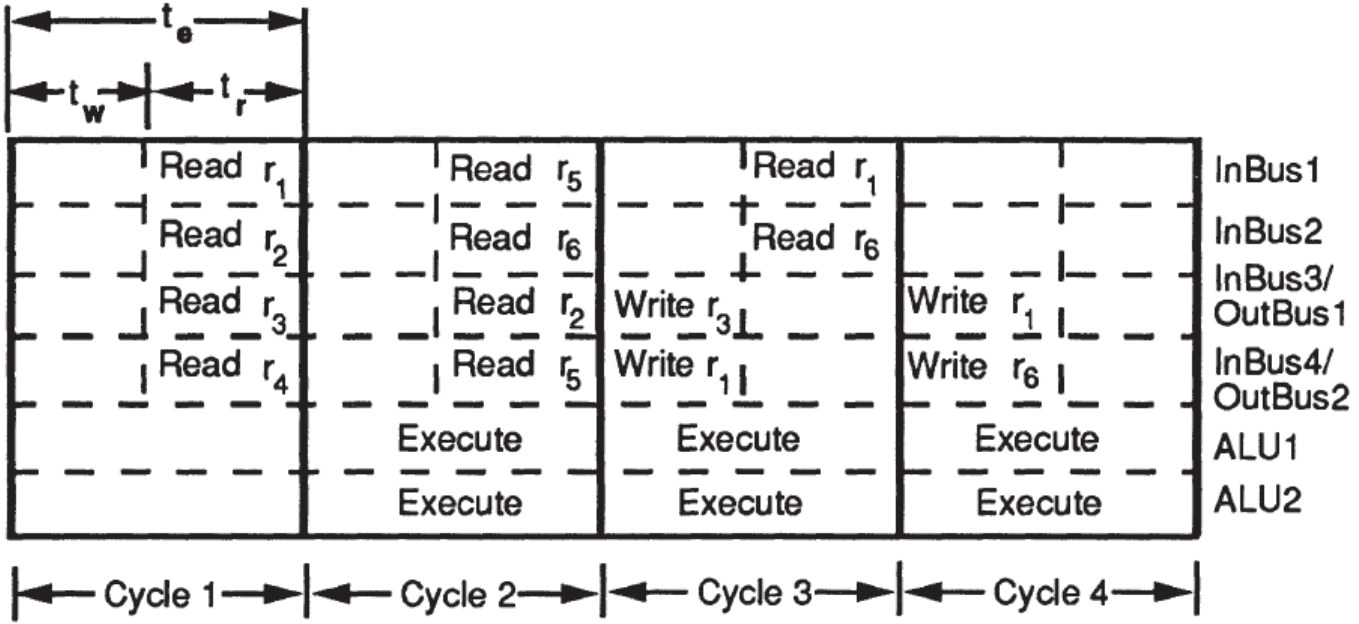


Figure 6: Insertion of latches at both the input and output ports of the functional units.

Datapath Architectures

→ **Figure 7**: Overlapping data transfer with functional-unit execution.



Datapath Architectures

→ The clock cycle consists of two minor cycles and the execution of an operation is spread across three consecutive clock cycles.

→ The input operands for an operation are transferred from the register files to the input latches of the functional units during the second minor cycle of the first cycle.

→ During the second cycle, the functional unit executes the operation and writes the result to the output latch by the end of the cycle.

→ The result is transferred to the final destination, the register file, during the first minor cycle of the third cycle. A two-phase non-overlapping clocking scheme is needed.

→ Both the input and output latches are controlled by one phase since the end of the read access and the end of operation execution occur simultaneously.

→ The other phase is used to control the write accesses to the register files.

Datapath Architectures

→ By overlapping the execution of operations in successive control steps, we can greatly increase the hardware utilization. The cycle period is reduced to

$$MAX(te_{,} t_r + t_w).$$

→ Moreover, the input and output networks can share some interconnection units.

→ For **example**, OutBus1 is merged with InBus3 and OutBus2 with InBus4 in Figure 6.

→ Thus, inserting input and output latches makes available more interconnection units for merging, which may simplify the datapath design.

→ By breaking the register-to-register transfers into micro-operations executed in different clock cycles, we achieve a better utilization of hardware resources.

→ However, this scheme requires binding algorithms to search through a larger number of design alternatives.

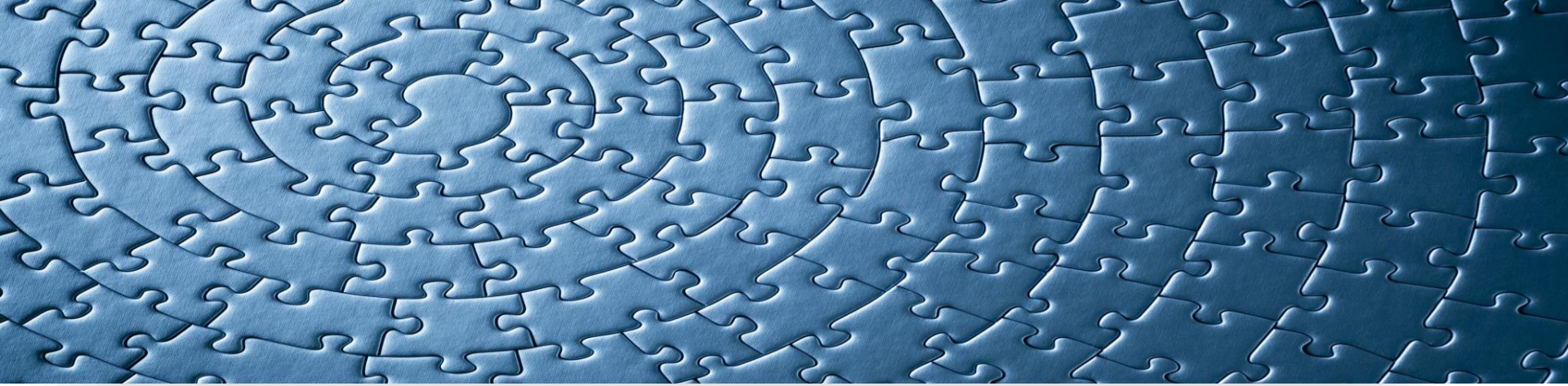
Datapath Architectures

→ Operator chaining was introduced before as the execution of two or more operations in series during the same control step.

→ To support operation chaining, links are needed from the output ports of some functional units directly to the input ports of other functional units.

→ In an architecture with a shared bus (e.g., Figure 6), this linking can be accomplished easily by using the path from a functional unit's output port through one of the buses to some other functional unit's input port.

→ Since such a path must be combinational, bypass circuits have to be added around all the latches along the path of chaining.



In the context of High-Level Synthesis (HLS) for digital circuit design, allocation tasks refer to the process of assigning available hardware resources to the operations identified in a behavioral description for implementation in a circuit. This step is crucial for optimizing the physical characteristics of the final hardware product, such as its size, power consumption, speed, and overall efficiency.

Allocation Problem

Allocation Problem

- → Scheduling, binding and allocation tasks are related to each other and need to be solved simultaneously for optimal design.
- → However, in practice, they are typically solved sequentially: first, scheduling is solved and then binding (more conventional) or first binding and then scheduling.
- → From a DFG perspective, scheduling is a temporal partitioning process, and binding is a spatial partitioning process.
- → Several subproblems are associated with a binding problem, such as operation to resource binding, binding values to be stored to instances of storage elements and binding data transfers to bus instances (i.e., functional unit binding, storage binding and interconnection binding).
- → Similarly, the allocation problem is divided into three tasks: functional unit, storage and interconnection allocation.
- → The optimization goals of the binding problem are to minimize total cost of functional units, register, bus driver and multiplexor, total interconnection length, critical path delay and power dissipation.
- → The sub-tasks of binding can be solved by following graph theoretic approaches, such as clique partitioning, circular-arc graph coloring or left edge algorithm.

Allocation Tasks: Unit Selection

→ A simple design model may assume that we have only one particular type of functional unit for each behavioral operation.

→ However, a real RT component library contains multiple types of functional units, each with different characteristics (e.g., functionality, size, delay and power dissipation) and each implementing one or several different operations in the register-transfer description.

→ For **example**, an addition can be carried out by either a small but slow ripple adder or by a large but fast carry look-ahead adder.

→ Furthermore, we can use several different component types, such as an adder, an adder/subtracter or an entire ALU, to perform an addition operation.

→ Thus, unit selection selects the number and types of different functional and storage units from the component library.

Allocation Tasks: Functional-Unit Binding

→ A basic requirement for unit selection is that the number of units performing a certain type of operation must be equal to or greater than the maximum number of operations of that type to be performed in any control step.

→ Unit selection is frequently combined with binding into one task called allocation.

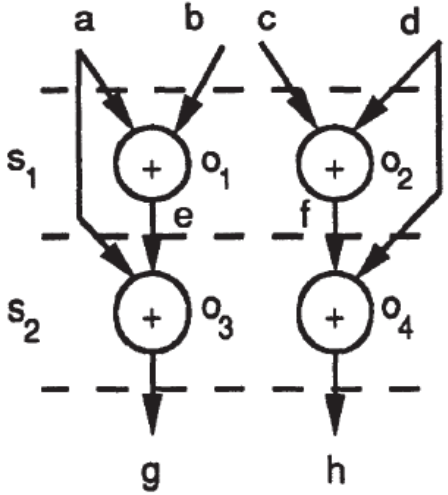
→ After all the functional units have been selected, operations in the behavioral description must be mapped into the set of selected functional units.

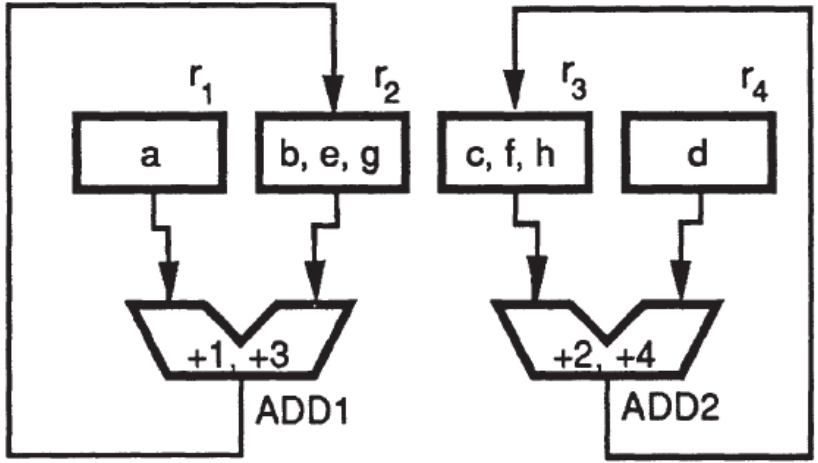
→ Whenever we have operations that can be mapped into more than one functional unit, we need a functional-unit binding algorithm to determine the exact mapping of the operations into the functional units.

→ For **example**, operations o_1 and o_3 in Figure 1 have been mapped into adder $ADD1$, while the operations o_2 and o_4 have been mapped into adder $ADD2$.

Allocation Tasks: Interconnection Binding

→ **Figure 1**: Mapping of behavioral objects into RT components.





Allocation Tasks: Storage Binding

→ Storage binding maps data carriers (e.g., constants, variables and data structures like arrays) in the behavioral description to storage elements (e.g., ROMs, registers and memory units) in the datapath.

→ Constants, such as coefficients in a DSP algorithm, are usually stored in a read-only memory (ROM).

→ Variables are stored in registers or memories. Variables whose lifetime intervals do not overlap with each other may share the same register or memory location.

- The lifetime of a variable is the time interval between its first value assignment (the first variable appearance on the left-hand side of an assignment statement) and its last use (the last variable appearance on the right-hand side of an assignment statement).

→ After variables have been assigned to registers, the registers can be merged into a register file with a single access port if the registers in the file are not accessed simultaneously.

→ Similarly, registers can be merged into a multiport register file as long as the number of registers accessed in each control step does not exceed the number of ports.

Allocation Tasks: Interconnection Binding

→ Every data transfer (i.e., a read or write) needs an interconnection path from its source to its sink.

→ Two data transfers can share all or part of the interconnection path if they do not take place simultaneously.

→ For **example**, in Figure 1, the reading of variable b in control step S_1 and variable e in control step S_2 can be achieved by using the same interconnection unit.

- However, writing to variables e and j, which occurs simultaneously in control step S_1, must be accomplished using disjoint paths.

→ The objective of interconnection binding is to maximize the sharing of interconnection units and thus minimize the interconnection cost, while still supporting the conflict-free data transfers required by the register-transfer description.

Allocation Tasks: Interdependence and Ordering

→ All the datapath synthesis tasks (i.e., scheduling, unit selection, functional unit binding, storage binding and interconnection binding) depend on each other. In particular, functional-unit, storage and interconnection binding are tightly related to each other.

→ For **example**, Figure 8 shows how both functional unit and storage binding affect interconnection allocation.

→ Suppose the eight variables (a through g) in the DFG of Figure 8(a) have been partitioned into four registers as follows: $r_1 \leftarrow \{a\}$, $r_2 \leftarrow \{b, e, g\}$, $r_3 \leftarrow \{c, j, h\}$, $r_4 \leftarrow \{d\}$.

→ Given two adders, $ADD1$ and $ADD2$, there are two ways of grouping the four addition operations, o_1, o_2, o_3, and o_4, in Figure 8(a) so that each group is assigned to one adder:

Case (1) $ADD1 \leftarrow \{o_1, o_4\}$, $ADD2 \leftarrow \{o_2, o_3\}$, or

Case (2) $ADD1 \leftarrow \{o_1, o_3\}$, $ADD2 \leftarrow \{o_2, o_4\}$.

Allocation Tasks: Interdependence and Ordering

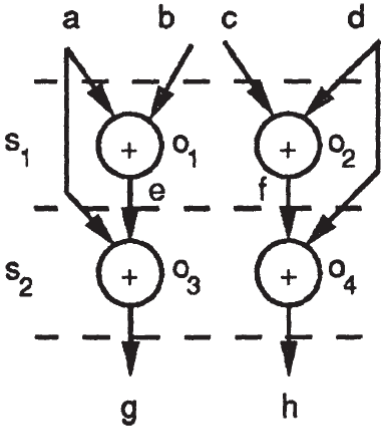
→ **Figure 8**: Interdependence of functional-unit and storage binding:

a) a scheduled DFG,

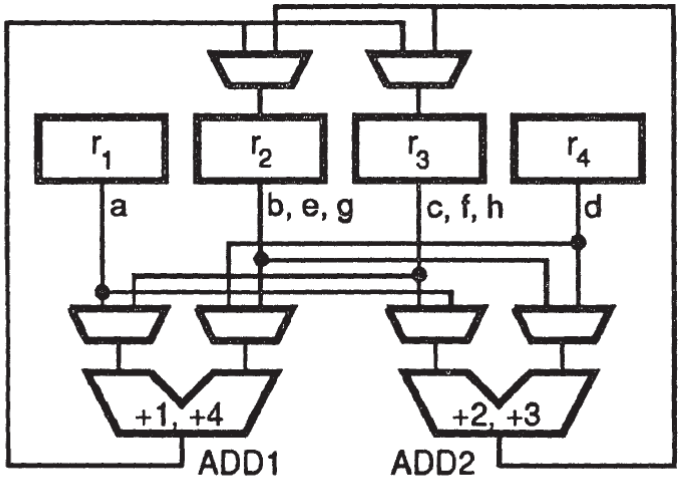
b) a functional-unit binding requiring six multiplexers,

c) improved design with two fewer multiplexers, obtained by register reallocation,

d) optimal design with no multiplexers, obtained by modifying functional-unit binding.



(a)



(b)

Allocation Tasks:

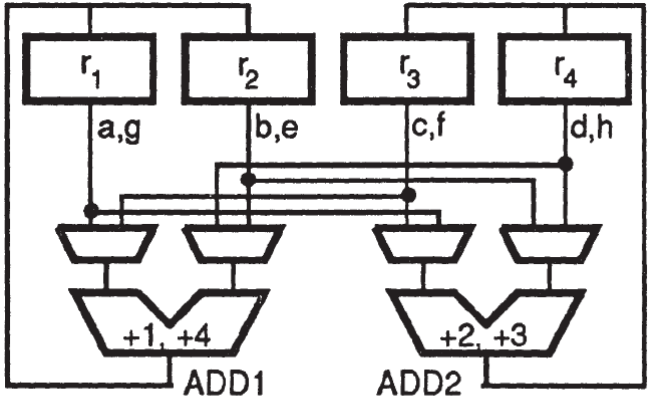
→ **Figure 8**: Interdependence of functional-unit and storage binding:

a) a scheduled DFG,

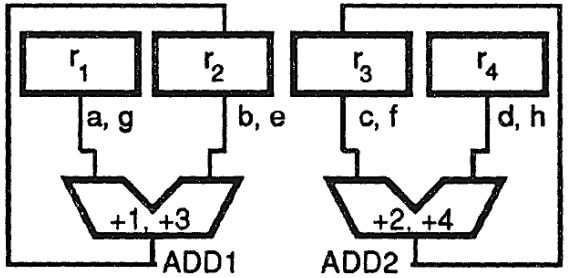
b) a functional-unit binding requiring six multiplexers,

c) improved design with two fewer multiplexers, obtained by register reallocation,

d) optimal design with no multiplexers, obtained by modifying functional-unit binding.



(c)



(d)

Allocation Tasks

→ For **example**, for the given register binding, we need six 2-to-1 multiplexers for unit interconnection in case (1) (Figure 8(b)).

- However, we can eliminate two 2-to-1 multiplexers (Figure 8(c)), by modifying the register binding to be:

$$r_1 \leftarrow \{a, g\}, r_2 \leftarrow \{b, e\}, r_3 \leftarrow \{c, j\}, r_4 \leftarrow \{d, h\}.$$

- If we then modify the functional-unit binding to case (2), no multiplexers are needed (Figure 8(d)).
- Thus, this design is optimal if the interconnection cost is measured by the number of multiplexers needed.

→ Clearly, both functional-unit and storage binding potentially affect the optimization achievable by interconnection allocation.

Allocation Tasks

→ The previous **example** also raises the issue of ordering among the allocation tasks.

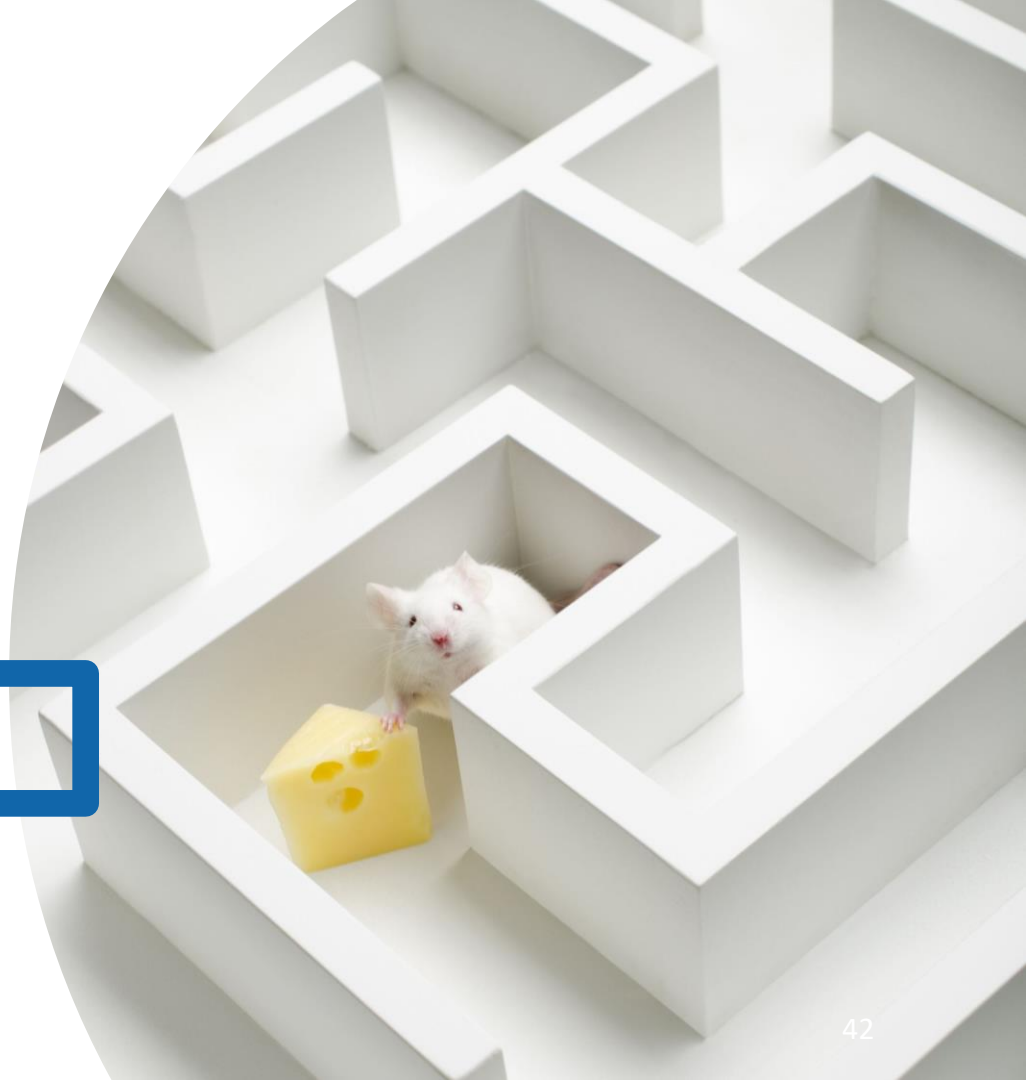
→ The requirements on interconnection become clear after both functional-unit and storage allocation have been performed.

→ Furthermore, functional-unit allocation can make correct decisions if storage allocation is done beforehand, and vice versa.

→ To break this deadlock situation, we choose one task ahead of the other.

→ Unfortunately, in such an ordering, the first task chosen cannot use the information from the second task, which would have been available had the second task been performed first.

Greedy Constructive Approaches



Greedy Constructive Approaches

→ A constructive algorithm starts with an empty datapath and builds the datapath gradually by adding functional, storage and interconnection units as necessary.

→ For each operation, it tries to find a functional unit on the partially designed datapath that is capable of executing the operation and is idle during the control step in which the operation must be executed.

→ In case there are two or more functional units that meet these conditions, we choose the one which results in a minimal increase in the interconnection cost.

→ On the other hand, if none of the functional units on the partially designed datapath meet the conditions, we add a new functional unit from the component library that is capable of carrying out the operation.

Greedy Constructive Approaches

→ Similarly, we can assign a variable to an available register only if its lifetime interval does not overlap with those of variables already assigned to that register.

→ A new register is allocated only when no allocated register meets the above condition.

→ Again, when multiple alternatives exist for assignment of a variable to a register, we select the one that minimally increases the datapath cost.

→ Figures 9(b)-(g) show several modified datapaths obtained by adding interconnection and/or functional units to the partial design of Figure 9(a).

→ Next, we discuss each case in this figure: (see next slide)

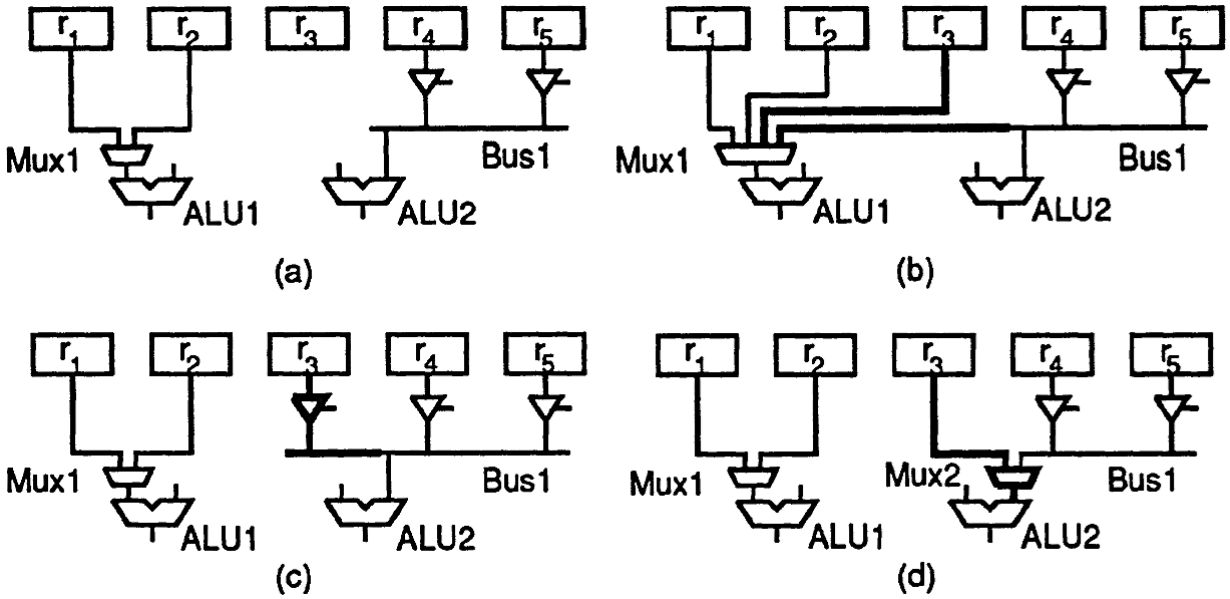
Greedy Constructive Approaches

1. In Figure 9(b) we add a new connection from each of r_3 and $Bus1$ to the left input of $ALU1$.
2. In Figure 9(c) we add a connection from r_3 to the right input of $ALU2$ through $Bus1$. Since the connection from $Bus1$ to the right input of $ALU2$ already exists, we add only a tri-state buffer.
3. Figure 9(d) is similar to Figure 8.9(c), except that we add the multiplexer $Mux2$ instead of a tri-state buffer.
4. In Figure 9(e) we add a new functional unit, $ALU3$, and a new connection from each of r_3 and $Bus1$ to the right input of $ALU3$.
5. Figure 9(f) is similar to Figure 9(e), except that we add the multiplexer $Mux2$ instead of a tri-state buffer.
6. In Figure 9(g) we merge $Muxl$ and $Bus1$ into a single bus, $Bus1$.

Greedy Constructive Approaches

→ Figure 9: Datapath construction:

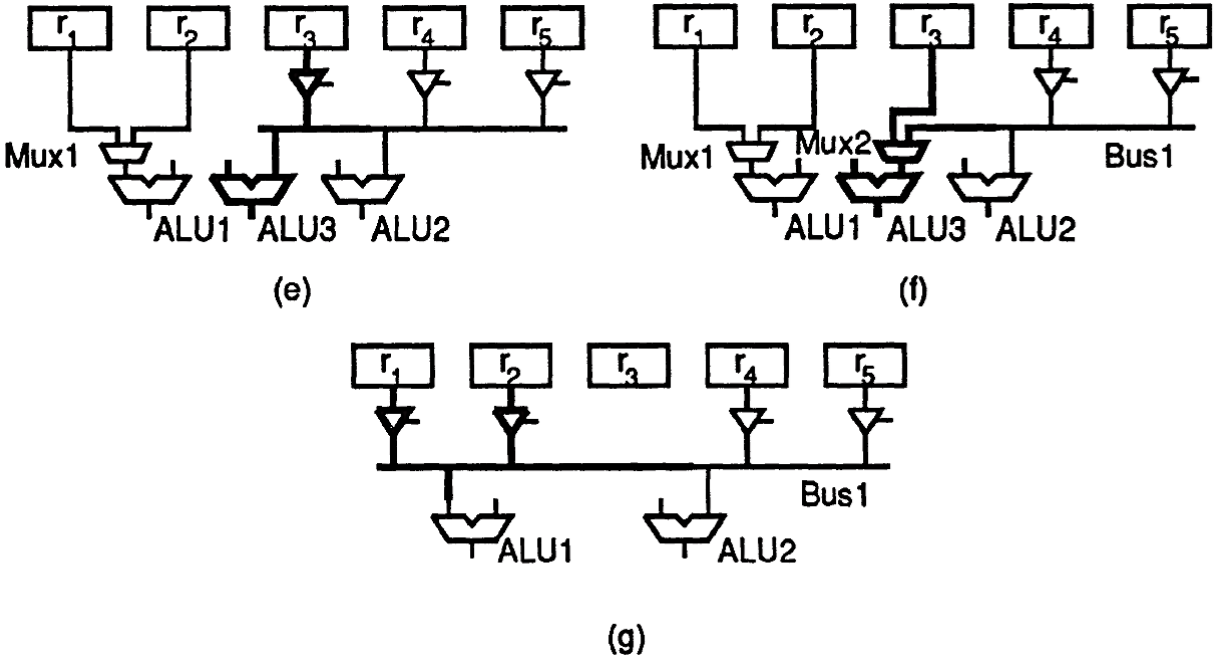
a) an initial partial design,
b) addition of two more inputs to a multiplexer,
c) addition of a tri-state buffer to a bus,
d) addition of a multiplexer to the input of a functional unit,
e) addition of a functional unit and a tri-state buffer to a bus,
f) addition of a functional unit and a multiplexer,
g) conversion of a multiplexer to a shared bus.



Greedy Constructive Approaches

→ Figure 9: Datapath construction:

a) an initial partial design,
b) addition of two more inputs to a multiplexer,
c) addition of a tri-state buffer to a bus,
d) addition of a multiplexer to the input of a functional unit,
e) addition of a functional unit and a tri-state buffer to a bus,
f) addition of a functional unit and a multiplexer,
g) conversion of a multiplexer to a shared bus.



Greedy Constructive Approaches

→ Interconnection allocation follows immediately after both the source and sink of a data transfer have been bound.

→ For **example**, the partial datapath in Figure 9(a) does not have a link between register r_3 and $ALU2$.

- Suppose a variable that is one of the inputs to an operation that has been assigned to $ALU2$ is just assigned to register r_3.
- Then, an interconnection link has to be established as shown by the bold wires in Figure 9(c).

→ Each interconnection unit that is required by a data transfer in the behavioral description contributes to the datapath cost.

→ For **example**, in Figure 9(b), two more connections are made to the left input port of $ALU1$, where a two-input multiplexer already exists.

- Therefore, the cost of this modification is the difference between the cost of a two-input multiplexer and that of a four-input multiplexer.
- However, at least two additional data transfers are now supported with this modification.

Greedy Constructive Approaches

→ Algorithm 1 describes the greedy constructive allocation method.

→ Let UBE be the set of Unallocated Behavioral Entities and $DP_{current}$ be the partially designed datapath.

→ The behavioral entities being considered could be variables that have to be mapped into registers, operations that have to be mapped into functional units, or data transfers that have to be mapped into interconnection units. $DP_{current}$ is initially empty.

→ The procedure $ADD(DP, ube)$ structurally modifies the datapath DP by adding to it the components necessary to support the behavioral entity ube.

→ The function $COST(DP)$ evaluates the area/performance cost of a partially designed datapath DP.

→ DP_{work} is a temporary datapath which is created in order to evaluate the cost C_{work} of performing each modification to $DP_{current}$.

Greedy Constructive Approaches

→ **Algorithm 1**: Constructive Allocation.

$DP_{current} = \phi$;
while $UBE \neq \phi$ **do**
  $LowestCost = \infty$;

  for all $ube \in UBE$ **do**
    $DP_{work} = \text{ADD}(DP_{current}, ube)$;
    $c_{work} = \text{COST}(DP_{work})$;
    if $c_{work} < LowestCost$ **then**
      $LowestCost = c_{work}$;
      $BestEntity = ube$;
    endif
  endfor

  $DP_{current} = \text{ADD}(DP_{current}, BestEntity)$;
  $UBE = UBE - BestEntity$;
endwhile

Greedy Constructive Approaches

→ Starting with the set UBE, the inner for loop determines which unallocated behavioral entity, $BestEntity$, requires the minimal increase in the cost when added to the datapath.

→ This is accomplished by adding each of the unallocated behavioral entities in UBE to $DP_{current}$ individually and then evaluating the resulting cost.

→ The procedure ADD then modifies $DP_{current}$ by incorporating $BestEntity$ into the datapath. $BestEntity$ is deleted from the set of unallocated behavioral entities.

→ The algorithm iterates in the outer while loop until all behavioral entities have been allocated (Le., $UBE = \phi$).

Greedy Constructive Approaches

→ In order to use the greedy constructive approach, we have to address two basic issues (The costs can be computed as explained before):

- the cost-function calculation,
- the order in which the unallocated behavioral entities are mapped into the datapath.

→ For **example**, the cost of converting the datapath in Figure 9(a) to the one in Figure 9(g) depends on the difference between the cost of one 2-to-l multiplexer and that of three tri-state buffers.

→ Since the buses of Figure 9(a) and Figure 9(g) are of different length, the two datapaths may also have different wiring costs.

→ The order in which unallocated entities are mapped into the datapath can be determined either statically or dynamically.

- In a static approach, the objects are ordered before the datapath construction begins. The ordering is not changed during the construction process.
- By contrast, in a dynamic approach no ordering is done beforehand.

Greedy Constructive Approaches

→ To select an operation or variable for binding to the datapath, we evaluate every unallocated behavioral entity in terms of the cost involved in modifying the partial datapath, and the entity that requires the least expensive modification is chosen.

→ After each binding, we revaluate the costs associated with the remaining unbound entities.

→ Algorithm 1 uses the dynamic strategy.

→ **Note**: In hardware sharing, a previously expensive binding may become inexpensive after some other bindings are done.

→ Therefore, a good strategy incorporates a look-ahead factor into the cost function.

→ That is, the cost of a modification to the datapath should be lower if it decreases the cost of other bindings done in the future.

Decomposition Approaches

Decomposition Approaches

→ The intuitive approach taken by the constructive method falls into the category of greedy algorithms.

→ Although greedy algorithms are simple, the solutions they find can be far from optimal.

→ In order to improve the quality of the results, some researchers have proposed a decomposition approach, where the allocation process is divided into a sequence of independent tasks; each task is transformed into a well-defined problem in graph theory and then solved with a proven technique.

→ While a greedy constructive approach like the one described in Algorithm 1 might interleave the storage, functional-unit, and interconnection allocation steps, decomposition methods will complete one task before performing another.

Decomposition Approaches: Clique Partitioning

→ The three tasks of storage, functional-unit and interconnection allocation can be solved independently by mapping each task to the well-known problem of graph clique-partitioning.

→ We begin by defining the clique-partitioning problem. Let $G = (V, E)$ denote a graph, where V is the set of vertices and E the set of edges.

→ Each edge $e_{i,j} \in E$ links two different vertices v_i and $v_j \in V$. A subgraph SG of G is defined as (SV, SE), where $SV \subseteq V$ and $SE = \{e_{i,j} | e_{i,j} \in E, v_i, v_j \in SV\}$.

- A graph is complete if and only if for every pair of its vertices there exists an edge linking them.
- A clique of G is a complete subgraph of G.

→ The problem of partitioning a graph into a minimal number of cliques such that each node belongs to exactly one clique is called clique partitioning.

→ The clique-partitioning problem is a classic NP-complete problem for which heuristic procedures are usually used.

Decomposition Approaches: Clique Partitioning

→ Two operations are compatible if they need resources of the same type and are not scheduled in the same clock cycle and thus can use the same resources.

→ To analyze compatibility of vertices/operations, a data structure called "compatibility graph" is useful.

→ This graph is defined as follows:

A resource compatibility graph $G_S^{com}(V, E)$ is an undirected graph whose vertex set $V = \{v_i, i = 1,2, \dots, N_v\}$ is in one-to-one correspondence with the operations and whose edge set $E = \{(v_i, vj), i = 1,2, \dots, Nv \text{ and } j = 1,2, \dots, Nv\}$ denotes the compatible vertex pairs.

→ An **example** of such a graph is shown in Fig. 10(b) for a scheduled DFG of Fig. 10(a), for two types of resources, multipliers and ALUs.

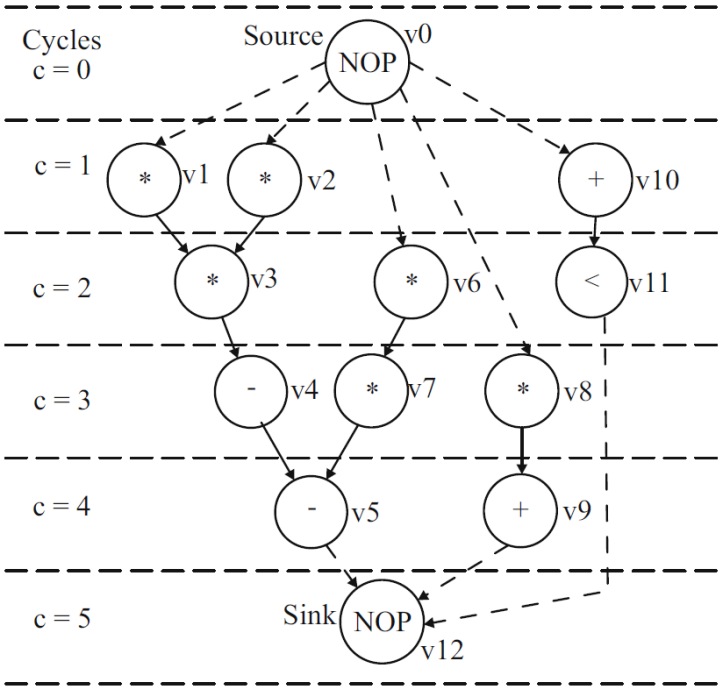
Decomposition Approaches: Clique Partitioning

→ **Figure 10**. DFG and corresponding compatibility graph and conflict graph for two types of resources

a) Scheduled sequencing data flow graph.

b) Compatibility graph.

c) Conflict graph



(a) Scheduled sequencing data flow graph

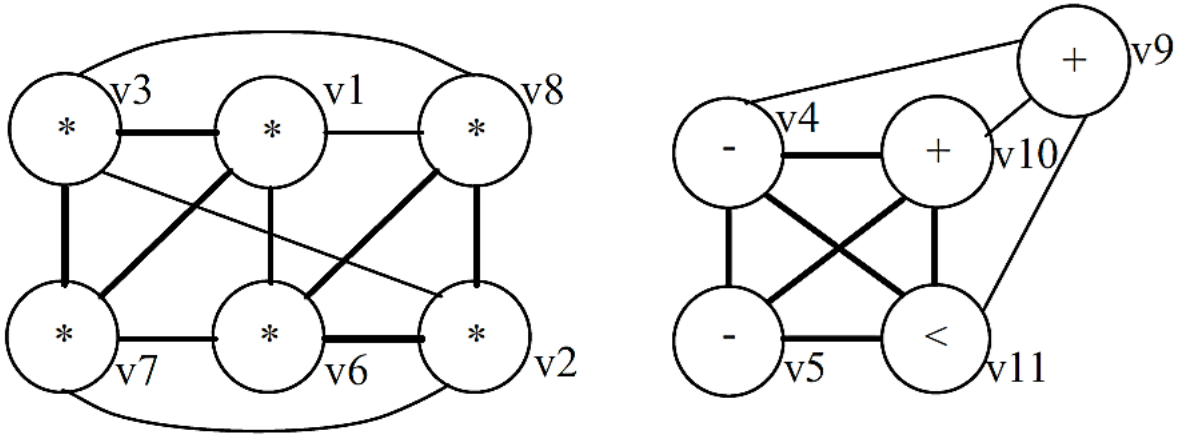
Decomposition Approaches: Clique Partitioning

→ **Figure 10**. DFG and corresponding compatibility graph and conflict graph for two types of resources

a) Scheduled sequencing data flow graph.

b) Compatibility graph.

c) Conflict graph



(b) Compatibility graph

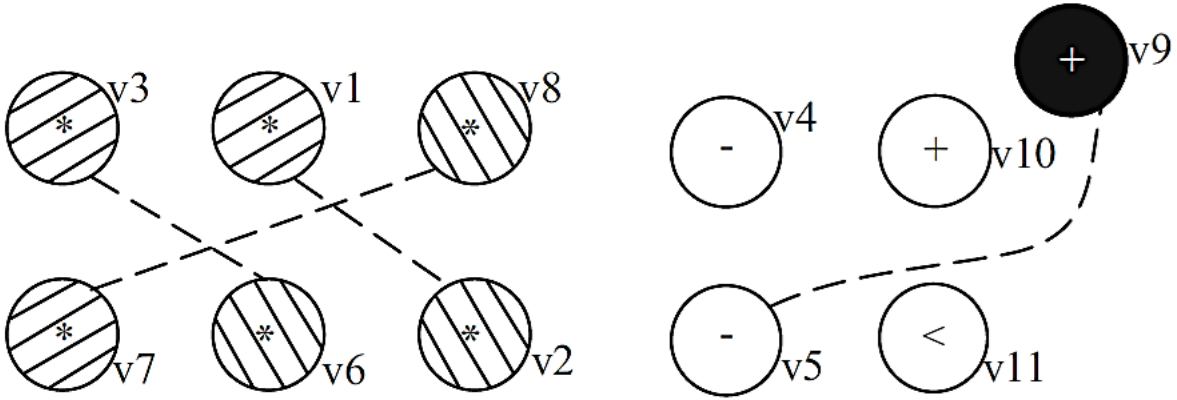
Decomposition Approaches: Clique Partitioning

→ Fig. 10 DFG and corresponding compatibility graph and conflict graph for two types of resources

a) Scheduled sequencing data flow graph.

b) Compatibility graph.

c) Conflict graph



(c) Conflict graph

Decomposition Approaches: Clique Partitioning

→ As it is evident from the figure, the graph has two disjoint components, the same as the type of resources.

→ A clique is a subset of vertices mutually connected by edges.

→ For **example**, in Fig. 10(b), the cliques are $\{v_1, v_3, v_7\}$, $\{v_2, v_6, v_8\}$, $\{v_4, v_5, v_{10}, v_{11}\}$ and $\{v_9\}$.

→ The optimal binding is then to partition the compatibility graph into the minimum number of cliques; this number is called the clique cover number.

→ For the above **example**, the clique cover number is 4; thus, there is a need for four resources, two multipliers and two ALUs.

→ The clique partitioning problem is NP-complete and, hence the binding problem is also NP-complete.

Decomposition Approaches: Graph Coloring Approach

→ Two operations have a conflict if they are not compatible.

→ To analyze the conflicts of vertices/operations, a data structure called "conflict graph" is useful, which is defined as follows.

A resource compatibility graph $G_S^{com}(V, E)$ is an undirected graph whose vertex set $V = \{v_i, i = 1,2, \dots, N_v\}$ is in one-to-one correspondence with the operations and whose edge set $E = \{(v_i, vj), i = 1,2, \dots, Nv \; and \; j = 1,2, \dots, Nv\}$ denotes the compatible vertex pairs.

→ An **example** of such a graph is shown in Fig. 10(c) for the scheduled DFG of Fig. 10(a) for two types of resources, multipliers and ALUs.

→ In a conflict graph, an independent set is a subset of vertices that are not connected by edges representing a set of mutually compatible operations.

→ For **example**, in Fig. 10(c), the independent sets are $\{v_1, v_3, v_7\}$, $\{v_2, v_6, v_8\}$, $\{v_4, v_5, v_{10}, v_{11}\}$ and $\{v_9\}$.

Decomposition Approaches: Graph Coloring Approach

→ The optimal binding is then to color the conflict graph with a minimum number of colors; this number is called the chromatic number.

→ Each color corresponds to an instance of a resource.

→ In the above **example**, each graph can be colored by two colors; thus, there is a need for four resources: two multipliers and two ALUs.

→ The graph coloring problem is NP-complete; hence, the binding problem is also NP-complete.

→ The conflict graph and compatibility graph are complementary to each other.

→ The choice between the use of compatibility is driven by the type of circuit; the graph which is sparse for a particular circuit is preferred for faster solution.

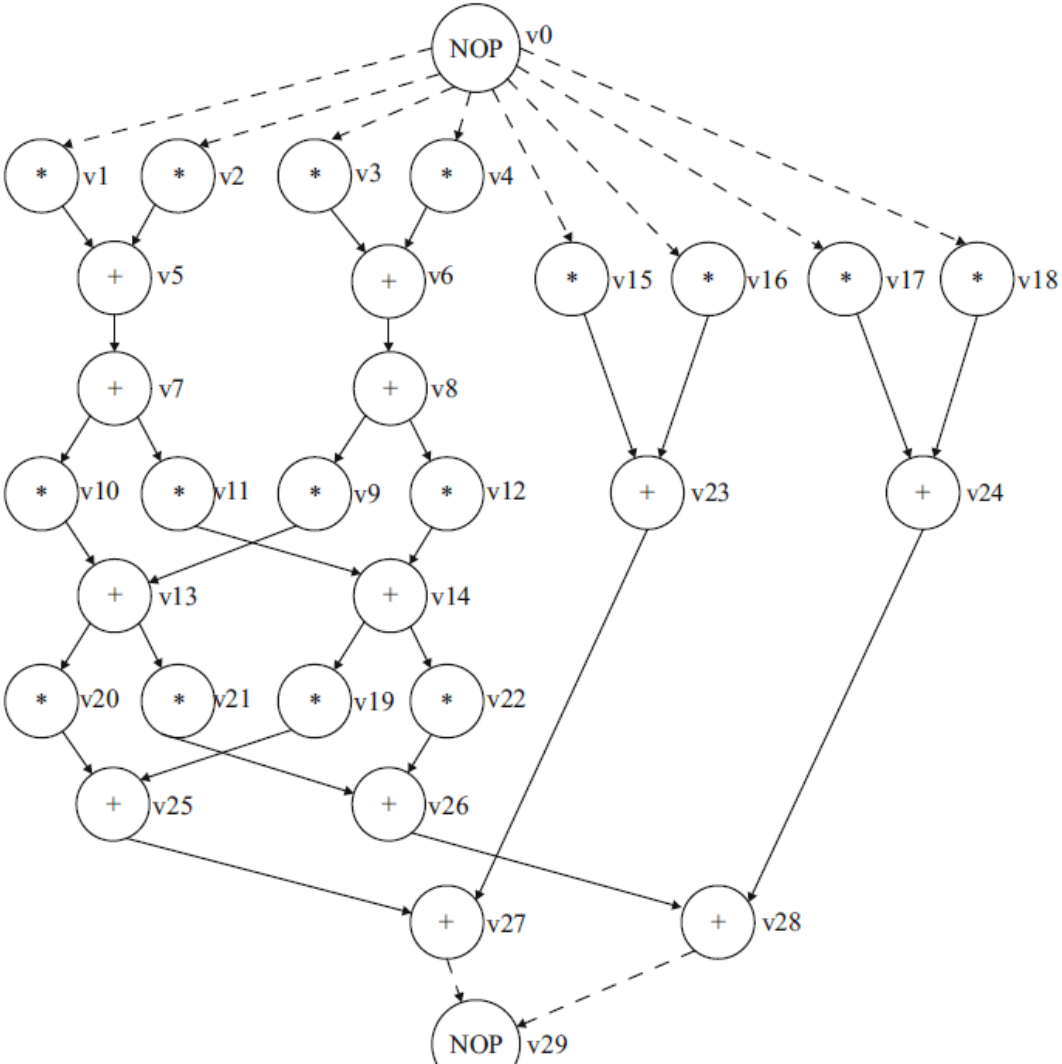
Some High-Level Synthesis Benchmarks

APPENDIX

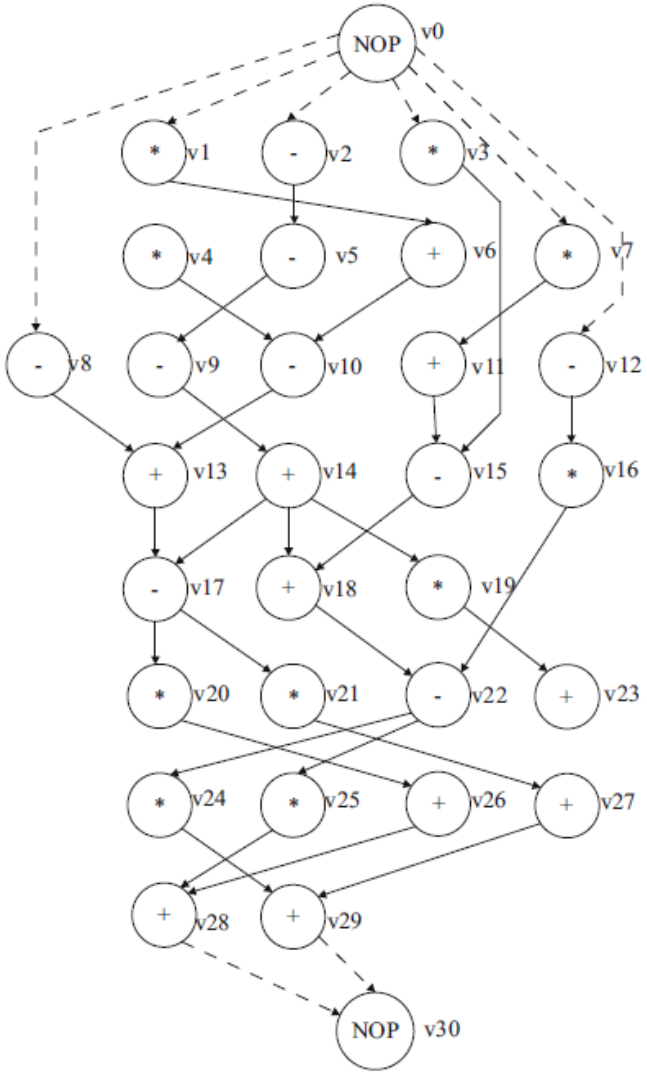
High-Level Synthesis Benchmarks

- Several high-level synthesis benchmarks are available in various forms to researchers to test the performance of their high-level synthesis-related research.
- The benchmarks can be obtained from various sources. In addition, random CDFGs can be generated using a pseudorandom graph generator presented in literature.
- Graphical representations of selected DSP benchmarks are provided in the following slides that researchers can express in different intermediate forms for their research:
  - autoregressive filter (ARF)
  - band-pass filter (BPF)
  - discrete cosine transformation (DCT) filter
  - discrete wavelet transformation (DWT)
  - elliptic wave filter (EWF)
  - fast Fourier transformation (FFT)
  - finite impulse response (FIR) filter
  - inverse discrete cosine transformation (IDCT) filter
  - MPEG motion vectors (MMV)
  - wave digital filter (WDF).

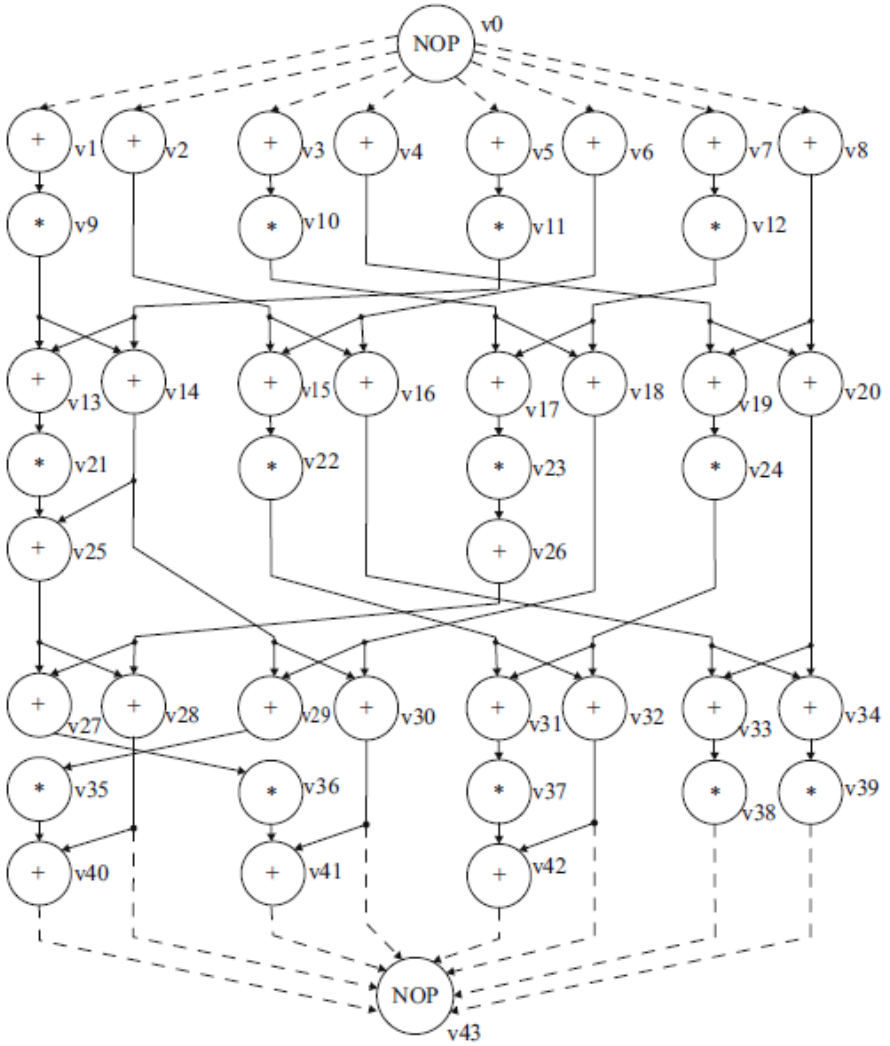
Auto regressive filter (ARF)



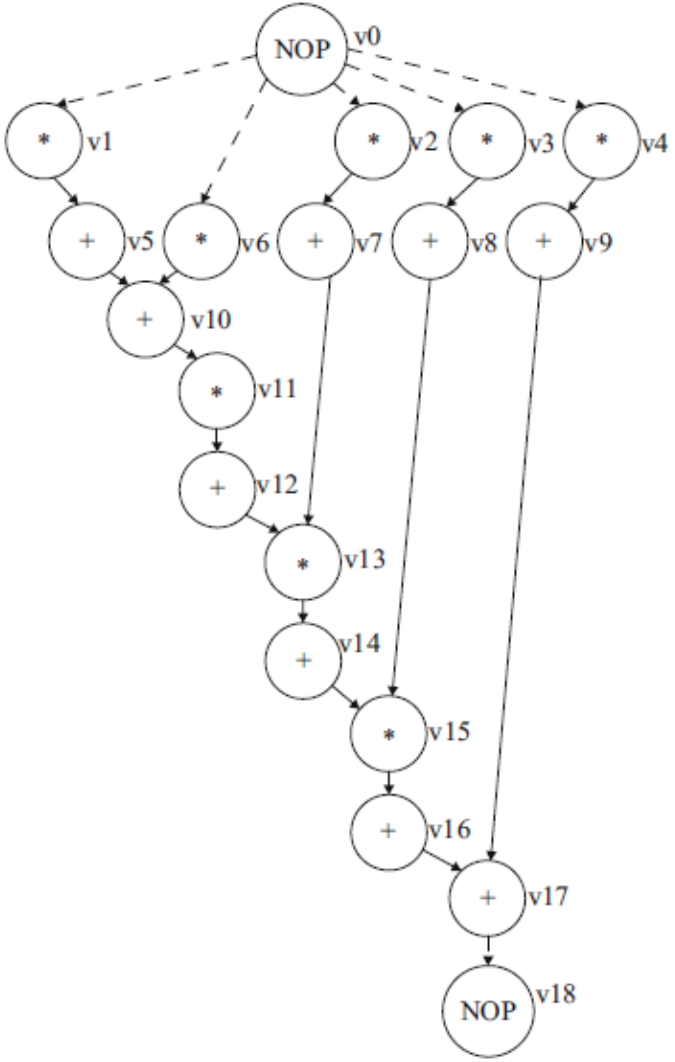
Band-pass filter (BPF)



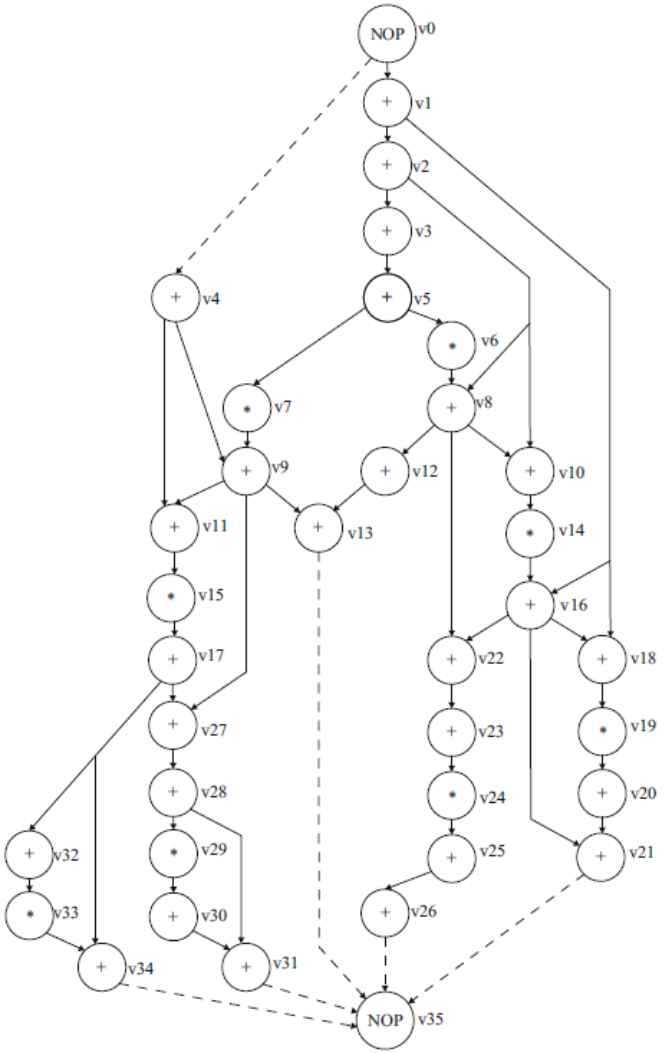
Discrete cosine transformation (DCT)



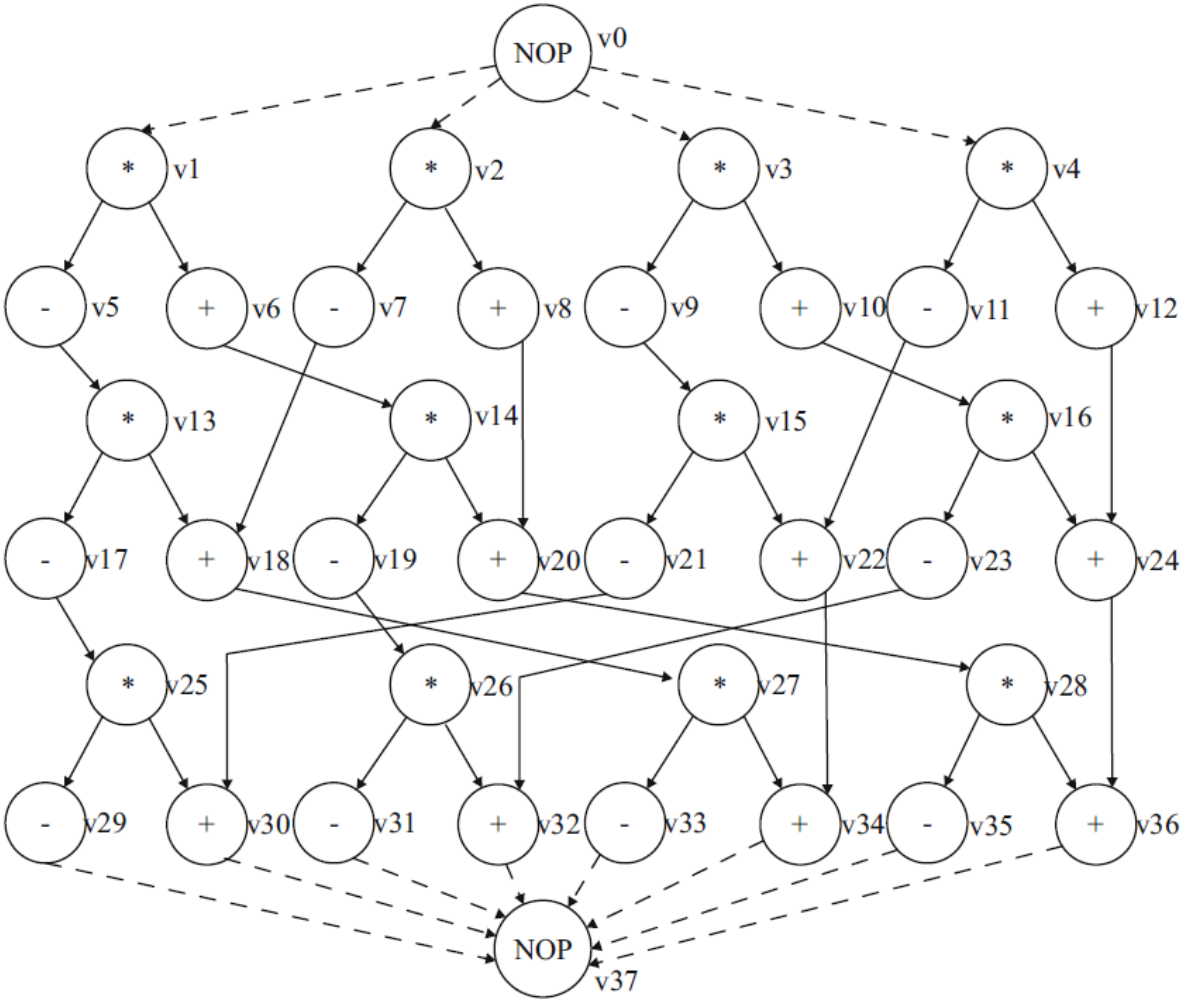
Discrete wavelet transformation (DWT)



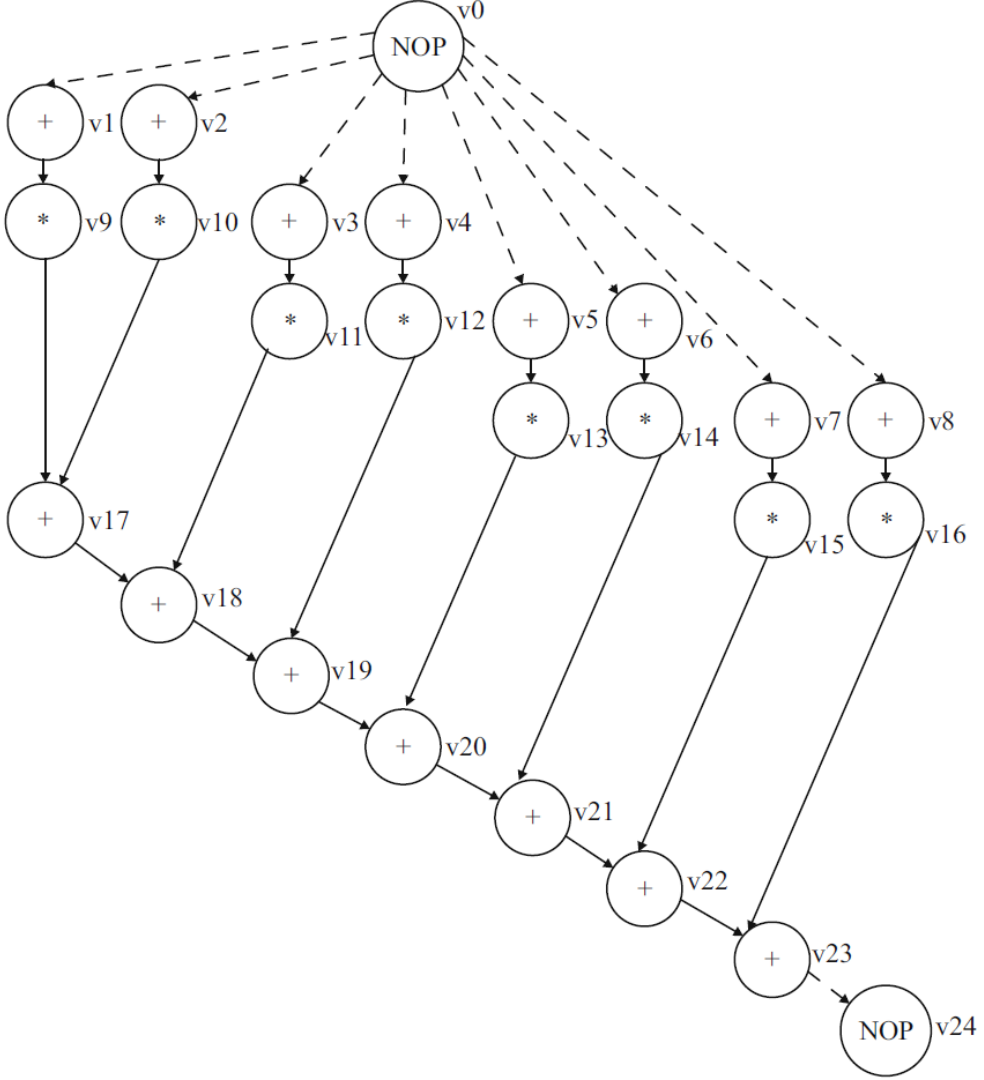
Elliptic wave filter (EWF)



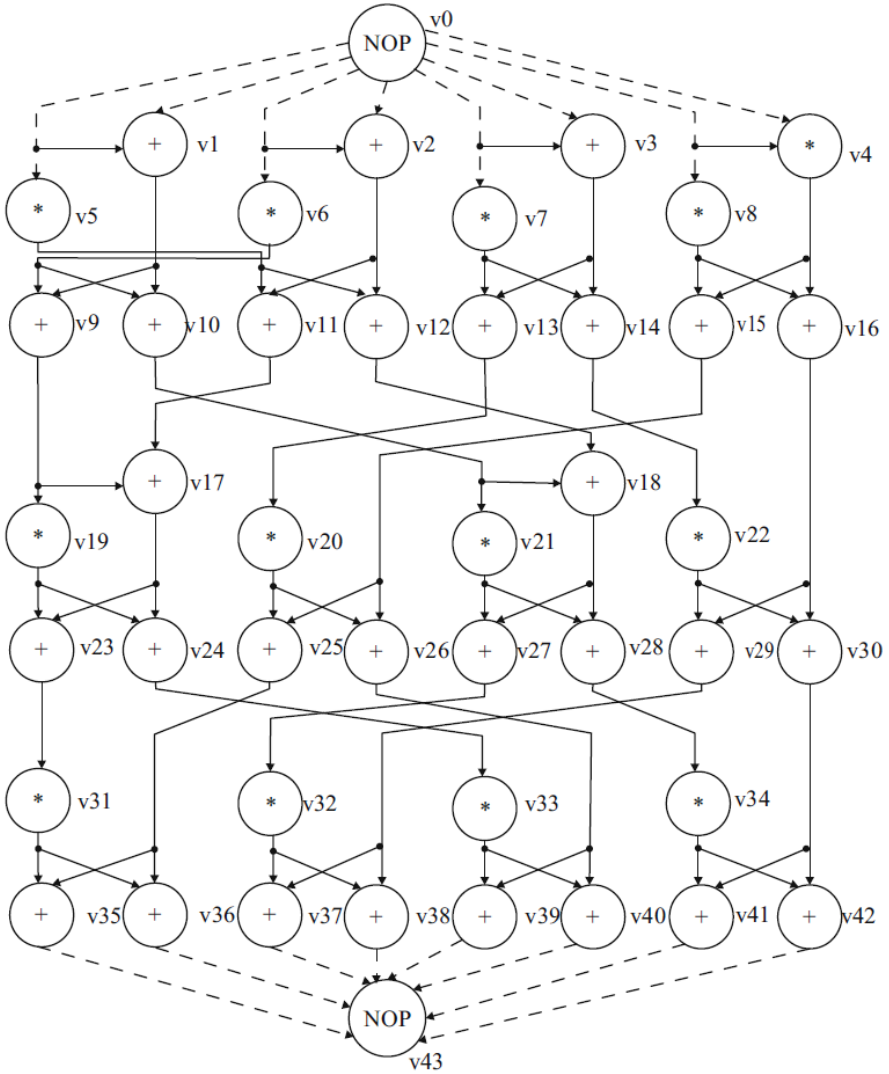
Fast Fourier transformation (FFT)



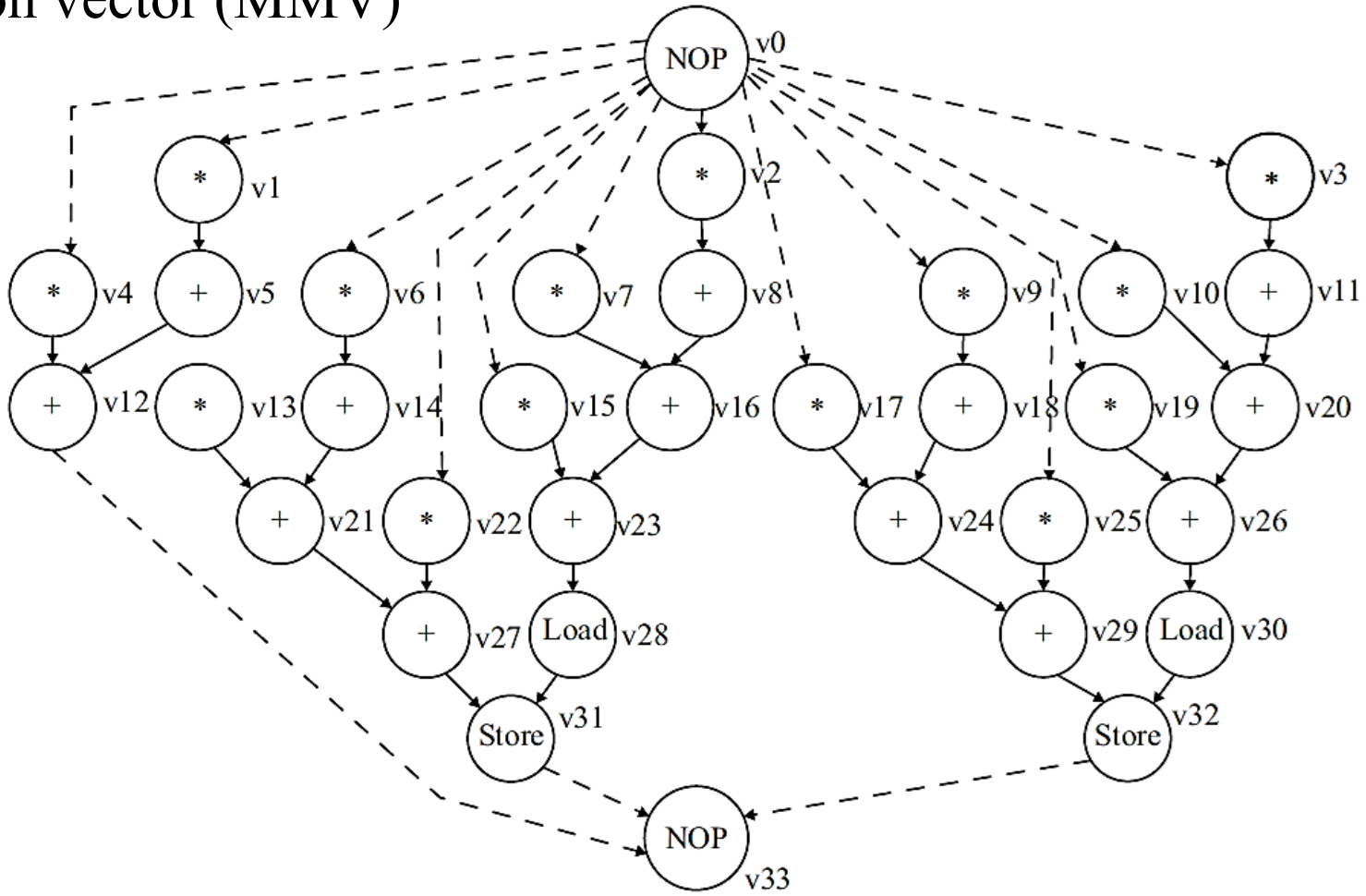
Finite impulse response filter (FIR)



Inverse discrete cosine transformation (IDCT)



MPEG motion vector (MMV)



Wave digital filter (WDF)